

Pratik Refactor Teknikleri ve Entropi Azaltma: Yazılım Sistemlerinde Bilgi Geometrisinin Sadeleştirilmesi

Ömer Faruk Yavuz

Temmuz 2026

Giriş

Yazılım geliştirme literatüründe yeniden düzenleme (refactoring) teknikleri çoğunlukla kod kalitesi ve okunabilirlik bağlamında ele alınır. Bu çalışma, söz konusu tekniklere daha derin bir işlev atfetmektedir: yeniden düzenleme pratikleri, sistemdeki belirsizliği, tekrarları, gizli bağımlılıkları ve kontrolsüz varyasyonları azaltan birer **entropi azaltma** (entropy reduction) mekanizmasıdır. Başka bir ifadeyle bu pratikler, sistemin bilgi geometrisini sadeleştirme sürecinin araçlarıdır.

Bu bakış açısının önemi, yapay zekâ destekli geliştirme çağında belirginleşmektedir. Yapay zekâ araçları, kod tabanındaki örüntülerden, isimlendirmelerden ve yapısal sinyallerden hareketle üretim yapar. Dolayısıyla aşağıda incelenen teknikler yalnızca kodu güzelleştirmek için uygulanmaz; bunlar aynı zamanda hem insan geliştiricinin hem de yapay zekânın kod tabanını daha doğru anlamasını sağlayan yapısal iyileştirmelerdir.

Çalışma iki bölümden oluşmaktadır. Birinci bölüm, sekiz temel yeniden düzenleme tekniğini tanımlamakta ve her birinin makine okunabilirliği üzerindeki etkisini incelemektedir. İkinci bölüm, bu teknikleri entropi azaltma çerçevesinde birleştirmekte ve bilgi geometrisinin sadeleştirilmesi kavramını tartışmaktadır.

Temel Yeniden Düzenleme Teknikleri

Sabit Değerlerin Merkezileştirilmesi

Kod içine doğrudan yazılmış ve özel anlam taşıyan dizgi değerleri (magic strings), kısa vadede pratik görünen fakat sistem büyüdükçe risk üreten bir kullanım biçimidir. Örneğin bir durum karşılaştırmasının doğrudan bir metin değeriyle yapılması yaygın bir örnektir:

```
if (status === "completed") {  
    ...  
}
```

Bu kullanımda aynı değer farklı dosyalarda tekrar eder, yazım hatası riski taşır ve değer değiştiğinde tüm proje genelinde arama yapmayı gerektirir. Daha sağlıklı yaklaşım, değeri merkezi ve anlamlı bir yapıya taşımaktır:

```
if (status === TASK_STATUS.COMPLETED) {  
    ...  
}
```

Bu dönüşüm yapay zekâ açısından da anlamlıdır; çünkü model artık rastgele bir dizgi değil, adlandırılmış bir alan kavramı görür. Sabit değerlerin merkezileştirilmesi şu kazanımları sağlar:

- tekrarları azaltır,
- yazım hatası riskini düşürür,
- alan kavramlarını görünür kılar,
- yeniden düzenlemeyi kolaylaştırır,
- modele daha net anlamsal sinyal verir.

Entropi perspektifinden bakıldığında bu teknik, dağınık dizgi değerlerini merkezi alan sembollerine dönüştürür; aynı kavramın tek bir noktadan temsil edilmesini sağlayarak sistemdeki rastgeleliği azaltır.

Yardımcı Fonksiyon Ayrıştırma

Yardımcı fonksiyon ayrıştırma (helper extraction), tekrar eden veya karmaşık küçük işlemleri ayrı ve adlandırılmış fonksiyonlara taşımaktır. Örneğin bir bileşen içinde sürekli tekrarlanan tarih biçimlendirme işlemi:

```
const formattedDate = new Date(value).toLocaleDateString(...);
```

anlamlı bir yardımcı fonksiyona taşındığında:

```
formatEventDate(value)
```

yalnızca kod kısalmaz; niyet açık hale gelir. Model açısından bu dönüşüm değerlidir; çünkü model artık düşük seviyeli işlem detayını değil, anlamı açık bir fonksiyon adını okur. Bu farkın özü şu şekilde ifade edilebilir:

Kod, ne yaptığını saklamaz; fonksiyon adıyla açıklar.

Entropi çerçevesinde yardımcı fonksiyon ayrıştırma, bileşen veya servis içinde gömülü kalan işlem detaylarını adlandırılmış niyetlere dönüştürür.

Akıllı–Sunum Bileşeni Ayrımı

Akıllı–sunum ayrımı (smart/dumb separation), bileşenleri iki kategoriye ayırma yaklaşımıdır. Akıllı bileşen; veri çekme, durum bağlama, eylem tetikleme ve iş akışını yönetme sorumluluğunu taşır. Sunum bileşeni ise yalnızca aldığı veriyi gösterir ve kullanıcı etkileşimlerini dışarıya olay olarak bildirir.

Bu ayrım özellikle arayüz katmanı ağırlıklı projelerde önemlidir. Akıllı bileşen tipik olarak şunları yapar:

- merkezi durum deposundan veri okur,
- dış sistem çağrısını tetikler,
- yükleme ve hata durumlarını yönetir,
- alt bileşene işlenmiş veri iletir.

Sunum bileşeni ise şunlarla sınırlıdır:

- kendisine verilen veriyi görüntüler,
- kullanıcı etkileşimlerini olay olarak yayınlar,
- iş mantığı bilmez,
- küresel durum bilmez.

Bu ayrım modelin işini belirgin biçimde kolaylaştırır; çünkü model bir bileşene baktığında onun sorumluluğunu hızla kavrar. Ayrımın yokluğunda bileşenler şişer; arayüz, durum, dış iletişim ve mantık aynı dosyada yığılır. Bu yığılma hem insanı hem modeli yavaşlatır. Entropi açısından bu teknik, sistemin sorumluluk geometrisini netleştirir: akışı yöneten yapı ile görüntüleyen yapı birbirinden ayrılır.

Facade (Cephe) Deseni

Facade deseni, karmaşık bir alt sistemi daha basit ve temiz bir arayüz arkasına saklama yöntemidir. Arayüz katmanında facade, bileşen ile durum yönetimi ve servis katmanları arasında konumlanır. Böylece bileşen şu detayları bilmek zorunda kalmaz:

- hangi eylemin tetiklendiği,
- hangi seçicinin kullanıldığı,
- hangi yan etki yöneticisinin hangi dış sistemi çağırdığı,
- yükleme durumunun nereden geldiği,
- hata durumunun nasıl yönetildiği.

Bileşen yalnızca facade ile iletişim kurar:

```
this.orderFacade.loadSummary(orderId);
this.summary$ = this.orderFacade.summary$;
```

Bu yaklaşım sistemi sadeleştirir; bileşenin bilişsel yükü azalır ve model için daha net bir mimari oluşur. Desenin iletmediği temel mesaj şudur:

Bileşen detayları bilmez; yalnızca niyeti ifade eder.

Entropi çerçevesinde facade ayrımı, karmaşık iç yapıyı sade bir kullanım yüzeyine dönüştürür; hem mimari sınırları netleştirir hem de modelin bileşen sorumluluğunu doğru anlamasını sağlar.

İzole Test Ortamı

İzole test ortamı (isolated test rig), belirli bir özelliği veya akışı ana uygulamadan bağımsız biçimde sınamaya yarayan kontrollü ortamdır. Uzun süren bir üretim akışının, bir arşivleme davranışının, bir olay akışı ayrıştırma sürecinin veya farklı sunucu yanıt senaryolarının izole bir yüzeyde denenmesi bu kapsama girer.

Bu yaklaşımın kazanımları şunlardır:

- belirli akışlar ana uygulama kullanılmadan sınanır,
- sınır durumları hızla denenir,
- taklit verilerle tekrarlanabilir senaryolar oluşturulur,
- hata ayıklama süresi kısalmır,
- gerileme riski daha erken görünür hale gelir.

Model açısından izole test ortamı kritik bir işlev görür: model yalnızca kodu değiştirmekle kalmaz, değişikliğin belirli bir ortamda nasıl doğrulanacağını da kavrayabilir. Böylece “önerilen değişiklik gerçekten bu akışı düzeltiyor mu” sorusu yanıtlanabilir hale gelir. Test ortamının yokluğunda model teorik olarak doğru kod yazabilir; fakat akışın pratikte çalışıp çalışmadığını doğrulamak zorlaşır. Entropi açısından bu teknik, dağınık üretim davranışını kontrollü bir sınama yüzeyine taşır ve model için güvenilir bir geri bildirim döngüsü oluşturur.

Uç Nokta Sınama Yüzeyi

Uç nokta sınama yüzeyi, uygulamadaki sunucu uç noktalarını doğrudan sınamaya yarayan özel bir arayüzdür. Bu yüzey tipik olarak şu soruları yanıtlamak için kullanılır:

- sunucu gerçekten yanıt veriyor mu,
- istek yükü doğru mu,
- yanıt formatı beklenen biçimde mi,
- kimlik doğrulama bilgileri doğru iletiliyor mu,
- hata durumları nasıl dönüyor,
- problem istemciden mi yoksa sunucudan mı kaynaklanıyor.

Bu yüzeyin temel değeri, problemi izole etmesidir. Normal kullanıcı ekranında görülen bir hata; arayüz durumundan, form doğrulamasından, istek eşlemesinden, sunucu uç noktasından, kimlik doğrulamasından, ağ katmanından veya yanıt ayrıştırmasından kaynaklanıyor olabilir. Sınama yüzeyi bu katmanların bir kısmını devre dışı bırakır ve problemin kaynağı daha hızlı bulunur. Model açısından da bu yüzey faydalıdır; çünkü model bir hata üzerinde çalışırken uç nokta davranışını ayrı bir sınama yüzeyinden doğrulayabilir.

Akış Ayırıştırıcısının İzolasyonu

Sunucudan istemciye sürekli olay akışı gönderen mekanizmalar (örneğin sunucu kaynaklı olay akışları), uzun süren işlemlerde yaygın olarak kullanılır. Bu akışların ayırıştırma mantığı doğrudan bileşen içinde kaldığında bileşen karmaşıklaşır: bileşen hem arayüzü yönetir, hem akışı okur, hem olay satırlarını ayırıştırır, hem hataları ele alır, hem de durumu günceller.

Daha sağlıklı yaklaşım, akış ayırıştırıcısını ayrı bir yardımcı yapı veya servis içine almaktır:

```
parseStreamLine(line)
parseStreamEvent(rawEvent)
requestGenerateStream(payload)
```

Bu ayırıştırma şu kazanımları sağlar:

- akış ayırıştırma mantığı test edilebilir hale gelir,
- bileşen sadeleşir,
- sınır durumları daha kolay yönetilir,
- hata ayıklama kolaylaşır,
- model hangi kodun ayırıştırma, hangi kodun arayüz sorumluluğu taşıdığını net görür.

Bu teknik özünde bir sınır çizme örneğidir: akış protokolü ile arayüz davranışı birbirinden ayrılmış olur.

Durum Yönetiminin Netleştirilmesi

Merkezi durum yönetimi sistemleri güçlüdür; fakat kötü kullanıldıklarında karmaşık bir teknik borç kaynağına dönüşebilirler. Durum netleştirme, uygulamadaki durum alanlarının anlamını, kaynağını ve güncellenme biçimini belirgin hale getirmektir.

En yaygın problem, aynı verinin birden fazla yerde tutulmasıdır:

```
state.summary
state.current.summary
recordDetail.summary
recordDetail.manualSummary
```

Bu durumda geliştiricinin sorduğu temel soru şudur: gerçek kaynak hangisidir? Durum net değilse model de yanlış alanı güncelleyebilir; bir hatayı çözdüğünü sanırken gerçekte kullanılmayan bir alanı değiştirebilir.

Durum netleştirme çalışması şu ilkeleri kapsar:

- durum alanlarının tekil sorumluluğa sahip olması,
- aynı verinin birden fazla yerde tutulmaması,
- seçicilerin açık biçimde adlandırılması,

- eylemlerin niyet odaklı yazılması,
- indirgeyicinin yalnızca durum dönüşümü yapması,
- yan etkilerin ayrı bir katmanda toplanması,
- bileşenin karmaşık depo detaylarını doğrudan bilmemesi.

Bu yapı kurulduğunda tek yönlü akış izlenebilir hale gelir: eylem tetiklenir, yan etki katmanı dış çağrıyı yönetir, başarı veya başarısızlık eylemi döner, indirgeyici durumu günceller ve seçici bileşene işlenmiş veriyi taşır. Akış bu netlikte olduğunda hem hata ayıklama kolaylaşır hem de model değişiklikleri daha güvenli önerebilir.

Sözleşme ve Eşdeğerlik Testleri

Sözleşme testleri (contract tests), sistem parçaları arasındaki anlaşmaların bozulmadığını doğrular: yanıt formatları, olay yükleri veya bileşen arayüzleri bu kapsama girer. Eşdeğerlik testleri (parity tests) ise iki farklı gerçekleştirmenin aynı davranışı üretip üretmediğini denetler; özellikle geçiş, yeniden düzenleme veya eski-yeni sistem karşılaştırmalarında önemlidir.

Bu testler model açısından değerlidir; çünkü yapılan değişikliklerin davranışsal uyumu bozup bozmadığını daha net gösterirler. Entropi çerçevesinde sözleşme ve eşdeğerlik testleri, sistemin sınırlarını davranışsal düzeyde sabitleyen yapılardır.

Entropi Azaltma Çerçevesi

Yukarıda incelenen tekniklerin ortak paydası, tek tek kod iyileştirmelerinin ötesinde bir işlev taşımalarıdır:

Bu çalışmalar, sistemin entropisini azaltır.

Entropi azaltma; sistemdeki belirsizliği, tekrarları, gizli bağımlılıkları ve kontrolsüz varyasyonları azaltma işidir. Başka bir ifadeyle sistemin bilgi geometrisini sadeleştirme sürecidir.

Bir yazılım sisteminde her kavram, her bağımlılık, her durum alanı, her test ve her yan etki, sistemin bilgi geometrisini oluşturur. Teknik borç arttıkça bu geometri karmaşıklaşır: bağlantılar görünmez hale gelir, sınırlar bulanıklaşır, aynı kavram birden fazla yerde farklı biçimlerde temsil edilir ve sistemin anlam haritası bozulur.

Entropi azaltma çalışmaları bu geometriyi sistematik biçimde sadeleştirir:

- tekrar eden kavramlar merkezileştirilir,
- sorumluluklar ayrıştırılır,
- veri akışı görünür hale getirilir,
- yan etkiler izole edilir,
- testler davranışı sabitler,

- sözleşmeler sınırları tanımlar,
- isimlendirme alan anlamını taşır.

Her tekniğin entropi azaltıcı katkısı şu dönüşümlerle özetlenebilir: sabit değerlerin merkezileştirilmesi, dağınık değerleri merkezi alan sembollerine dönüştürür; yardımcı fonksiyon ayrıştırma, işlem detayını adlandırılmış niyete dönüştürür; facade ayrımı, karmaşık iç yapıyı sade kullanım yüzeyine dönüştürür; izole test ortamı, dağınık üretim davranışını kontrollü sınama yüzeyine taşır; akıllı-sunum ayrımı, akış yöneten yapı ile görüntüleyen yapıyı birbirinden ayırır; durum netleştirme, çoklu doğruluk kaynaklarını tekil kaynağa indirger; sözleşme testleri ise sınırları davranışsal düzeyde sabitler.

Yapay Zekâ Bağlamında Entropi Azaltmanın Değeri

Entropi azaltma çerçevesinin yapay zekâ çağındaki önemi, modelin çalışma biçiminden kaynaklanır. Model, olasılık tabanlı örüntü tamamlama ile üretim yapar; sistemdeki varyasyon ve belirsizlik arttıkça olası doğru yolların sayısı artar ve modelin isabet oranı düşer.

Entropi azaltılmış bir sistemde ise:

- aynı kavram tek biçimde temsil edildiği için model yanlış eşleştirme yapmaz,
- sorumluluklar ayrıştırıldığı için model değişikliği doğru katmana yerleştirir,
- yan etkiler izole edildiği için model etki alanını doğru öngörür,
- testler davranışı sabitlediği için model güvenilir geri bildirim alır,
- sözleşmeler tanımlı olduğu için model sınır ihlallerini fark edebilir.

Bu nedenle yeniden düzenleme, yapay zekâ destekli geliştirmede maliyet kalemi değil, yatırım kalemidir: entropisi düşürülmüş her sistem bölgesi, hem insan hem de makine için akıl yürütme kapasitesini artırır.

Sonuç

Bu çalışmada incelenen yeniden düzenleme teknikleri — sabit değerlerin merkezileştirilmesi, yardımcı fonksiyon ayrıştırma, akıllı-sunum ayrımı, facade deseni, izole test ortamı, uç nokta sınama yüzeyi, akış ayrıştırıcısının izolasyonu, durum netleştirme ve sözleşme testleri — klasik anlamda kod temizliği pratiklerinin ötesinde bir işlev taşımaktadır. Bu teknikler, sistemin bilgi geometrisini sadeleştiren, belirsizliği ve varyasyonu azaltan entropi azaltma mekanizmalarıdır.

Yapay zekâ destekli geliştirme çağında bu çerçevenin stratejik değeri artmıştır; çünkü modelin üretim kalitesi, sistemin sunduğu örüntü kalitesiyle sınırlıdır. Dolayısıyla temel sonuç şu şekilde ifade edilebilir:

Yeniden düzenleme, yalnızca kodu güzelleştirmek değildir; sistem üzerinde daha doğru düşünebilmek için bilgi geometrisini yeniden düzenleme işidir.